

Day 3 — Databases, Analytical SQL, and Caching for Applied AI Systems

Beginner-friendly summary

A production AI system normally uses several storage systems because each solves a different problem:

- **PostgreSQL** stores authoritative business data, workflows, model metadata, predictions, forecasts, and audit history.
- **DynamoDB** is useful for extremely high-throughput key-based access with predictable access patterns.
- **MongoDB** fits flexible document-shaped data whose structure changes frequently.
- **Redis** accelerates frequently accessed data but should rarely be the permanent source of truth.
- **Object storage** stores large immutable files such as documents, model artifacts, exports, and datasets.
- **Vector databases** store embeddings and support similarity search, but should not normally be the authoritative document catalog.

For finance and approval workflows, correctness usually matters more than raw flexibility. PostgreSQL is therefore commonly the default because it provides constraints, transactions, joins, and strong consistency.

1. Main alternatives and selection criteria

Technology	Best fit	Query pattern	Main strengths	Main limitations
PostgreSQL	Financial data, approvals, model registry, metadata, audit records	Joins, filters, aggregations, transactions	ACID transactions, constraints, SQL analytics, JSONB support	Horizontal scaling requires more planning
DynamoDB	High-volume key-value access, sessions, request state, event lookup	Partition key and optional sort key	Predictable latency, managed scaling, high availability	Weak fit for ad hoc joins and analytical queries
MongoDB	Flexible nested documents, changing schemas, content-oriented applications	Document lookup and aggregation pipelines	Flexible schema, natural JSON model	Cross-document consistency and relational analytics require care
Redis	Caching, rate limits, locks, short-lived state	Key lookup	Extremely low latency, TTL, atomic commands	Memory cost, invalidation complexity, usually not the source of truth

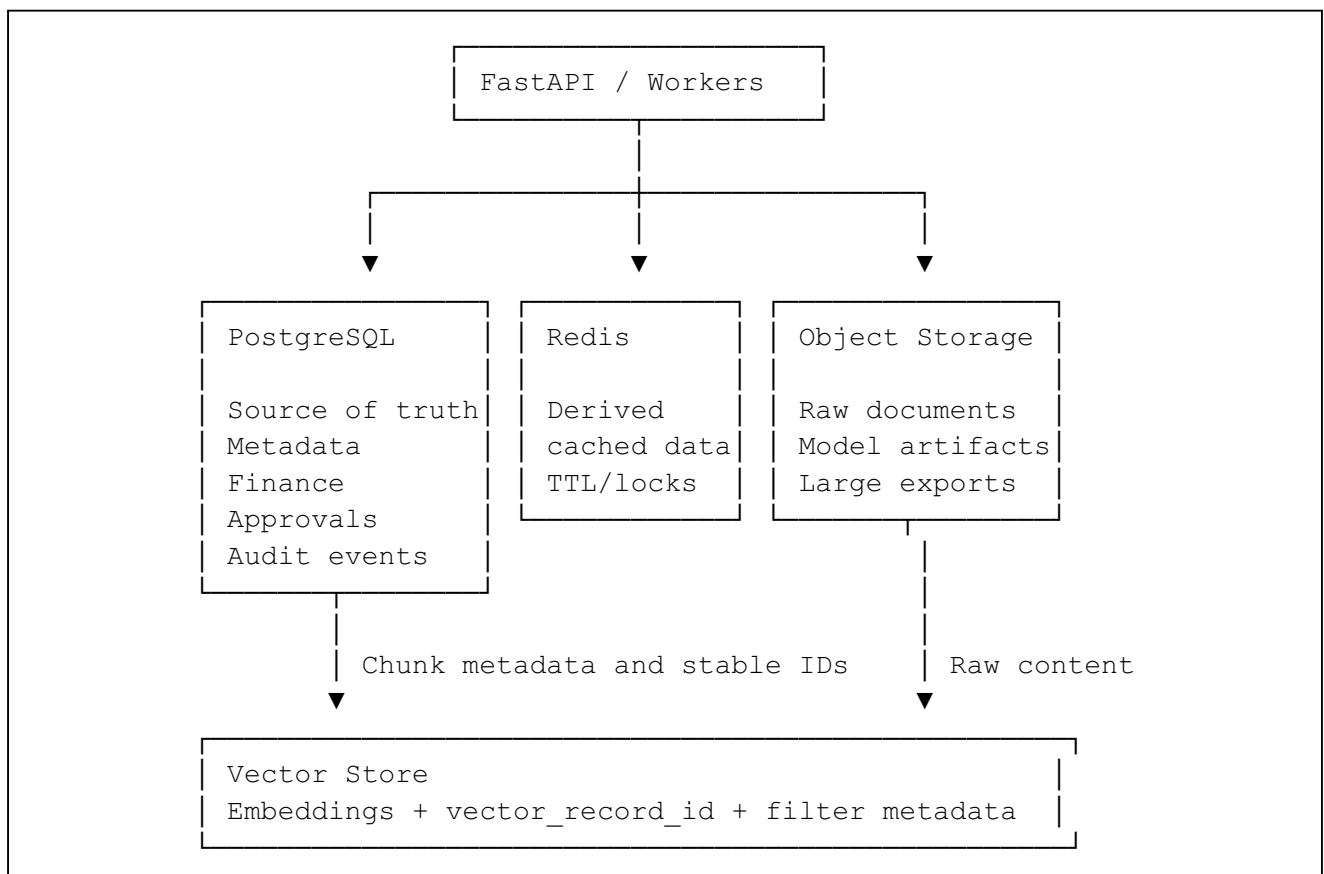
Object storage	Raw documents, datasets, model artifacts, large outputs	Retrieve by object key	Cheap, durable, scalable	Not suitable for transactional row-level queries
Vector store	Semantic retrieval	Approximate nearest-neighbour search	Fast embedding similarity search	Not a reliable source of truth for business metadata

Practical default

For many enterprise AI systems:

- Start with **PostgreSQL**.
- Add **Redis** only when measurements show repeated expensive reads.
- Store large files in **S3 or equivalent object storage**.
- Add a **vector database** for semantic search.
- Choose DynamoDB or MongoDB when their access patterns clearly fit better than relational modelling.

2. Reference architecture



The important separation is:

- PostgreSQL answers: **What exists, who owns it, what version is active, and what is its status?**
- Object storage answers: **Where are the original bytes?**

- Vector storage answers: **Which chunks are semantically similar?**
 - Redis answers: **Can a previously computed result be returned faster?**
-

3. SQL versus NoSQL at senior interview depth

3.1 PostgreSQL

Use PostgreSQL when the system requires:

- Multi-row transactions.
- Foreign-key relationships.
- Uniqueness and validation constraints.
- Complex filtering and reporting.
- Auditability.
- Approval or financial correctness.
- Ad hoc analysis.
- Strong consistency.

Typical AI-platform data:

- Model versions.
- Prompt versions.
- Prediction requests.
- Forecast runs.
- Evaluation results.
- Document metadata.
- Approval state.
- Billing and usage.
- Audit events.

PostgreSQL also supports `JSONB`, allowing limited schema flexibility without giving up relational guarantees.

Interview insight

Do not choose NoSQL merely because the system contains JSON. PostgreSQL can store JSON while retaining transactions, constraints, indexing, and joins.

3.2 DynamoDB

DynamoDB is strongest when access patterns are known before schema design.

Example access patterns:

- Fetch request state using `tenant_id + request_id`.
- Fetch all events for one workflow sorted by timestamp.
- Store high-volume conversation sessions.

- Retrieve an idempotency record by key.
- Store short-lived asynchronous job status.

A common key design could be:

```
PK = TENANT#{tenant_id}#JOB#{job_id}
SK = STATUS
```

or for event history:

```
PK = TENANT#{tenant_id}#JOB#{job_id}
SK = EVENT#{timestamp}
```

Important trade-off

In DynamoDB, you usually model around queries rather than normalizing entities first.

A relational designer asks:

| What entities and relationships exist?

A DynamoDB designer asks:

| Which exact reads and writes must execute efficiently?

Common failure

A team chooses DynamoDB and later needs:

- Arbitrary filtering.
- Complex finance reporting.
- Multi-entity reconciliation.
- Frequent joins.
- New access patterns not anticipated in the key design.

That often leads to scans, duplicated records, secondary-index proliferation, or an additional analytical database.

3.3 MongoDB

MongoDB fits data that naturally belongs together as one document.

For example:

```
{
  "conversation_id": "c-123",
  "tenant_id": "t-10",
  "messages": [
    {"role": "user", "content": "Explain forecast variance"},
    {"role": "assistant", "content": "..."}
  ],
  "metadata": {
```

```
    "channel": "web",  
    "language": "en"  
  }  
}
```

Embedding messages can be convenient when:

- The full conversation is normally loaded together.
- Individual messages are not updated independently.
- The document will not grow without bound.

Referencing messages separately is preferable when:

- Conversations can contain thousands of messages.
- Messages have independent lifecycle or compliance rules.
- Pagination by message is required.
- Concurrent updates are frequent.

Common failure

Unbounded arrays create increasingly large documents, expensive updates, and hot records.

4. Relational design principles

4.1 Entity identifiers

A practical pattern is:

- Use UUIDs for externally visible distributed entities.
- Use integer identity columns for internal high-volume fact tables when appropriate.
- Never use mutable business attributes such as email address or model name as the primary key.

Example:

```
id UUID PRIMARY KEY
```

A business identifier can still be unique:

```
UNIQUE (tenant_id, external_request_id)
```

4.2 Money

Never use `FLOAT` or `REAL` for financial values.

Use:

```
NUMERIC(18, 2)
```

because binary floating-point cannot exactly represent many decimal values.

For multi-currency systems, store:

```
amount NUMERIC(18, 2),  
currency_code CHAR(3)
```

A more advanced system may use different decimal scales for currencies or store minor units as integers.

4.3 Timestamps

Prefer:

```
TIMESTAMPZ
```

for real-world event timestamps.

Use a separate `DATE` for business dates such as:

- Posting date.
- Forecast period.
- Fiscal month.
- Settlement date.

Do not use timestamps to represent every business period.

4.4 Tenant isolation

Most tenant-owned tables should include:

```
tenant_id UUID NOT NULL
```

Uniqueness should usually be tenant scoped:

```
UNIQUE (tenant_id, invoice_number)
```

not globally scoped:

```
UNIQUE (invoice_number)
```

Indexes should frequently begin with `tenant_id` because most queries are tenant filtered.

5. Core production schemas

The following entities cover documents, predictions, forecasts, approvals, audit events, and model versions.

5.1 Model versions and prediction records

```
CREATE TABLE model_versions (  
  id                UUID PRIMARY KEY,  
  model_name        TEXT NOT NULL,  
  version           TEXT NOT NULL,  
  task_type         TEXT NOT NULL,  
  provider          TEXT NOT NULL,  
  artifact_uri      TEXT,  
  artifact_checksum TEXT,  
  configuration      JSONB NOT NULL DEFAULT '{} '::jsonb,  
  status            TEXT NOT NULL  
    CHECK (status IN (  
      'REGISTERED',  
      'VALIDATING',  
      'APPROVED',  
      'DEPLOYED',  
      'RETIRED'  
    )),  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
  UNIQUE (model_name, version)  
);  
  
CREATE TABLE prediction_requests (  
  id                UUID PRIMARY KEY,  
  tenant_id         UUID NOT NULL,  
  idempotency_key   TEXT NOT NULL,  
  model_version_id  UUID NOT NULL  
    REFERENCES model_versions(id),  
  input_payload     JSONB NOT NULL,  
  status            TEXT NOT NULL  
    CHECK (status IN (  
      'PENDING',  
      'RUNNING',  
      'SUCCEEDED',  
      'FAILED'  
    )),  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
  UNIQUE (tenant_id, idempotency_key)  
);  
  
CREATE TABLE predictions (  
  request_id        UUID PRIMARY KEY  
    REFERENCES prediction_requests(id),  
  output_payload    JSONB NOT NULL,
```

```

    latency_ms          INTEGER CHECK (latency_ms >= 0),
    input_tokens         INTEGER CHECK (input_tokens >= 0),
    output_tokens        INTEGER CHECK (output_tokens >= 0),
    completed_at         TIMESTAMPTZ NOT NULL
);

CREATE INDEX idx_prediction_requests_tenant_status_created
    ON prediction_requests (tenant_id, status, created_at DESC);

```

Design decisions

- The idempotency constraint prevents the same client request from producing duplicate predictions.
- The output is separated from request metadata because unsuccessful requests have no output.
- `model_version_id` preserves reproducibility.
- Input and output JSON are acceptable where shapes differ by model, but frequently queried attributes should become typed columns.

5.2 Forecast runs and values

```

CREATE TABLE forecast_runs (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    model_version_id  UUID NOT NULL
                        REFERENCES model_versions(id),
    as_of_date        DATE NOT NULL,
    horizon_periods   INTEGER NOT NULL
                        CHECK (horizon_periods > 0),
    frequency         TEXT NOT NULL
                        CHECK (frequency IN (
                            'DAILY',
                            'WEEKLY',
                            'MONTHLY',
                            'QUARTERLY'
                        )),
    status            TEXT NOT NULL
                        CHECK (status IN (
                            'PENDING',
                            'RUNNING',
                            'SUCCEEDED',
                            'FAILED'
                        )),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE forecast_values (
    forecast_run_id   UUID NOT NULL
                        REFERENCES forecast_runs(id)
                        ON DELETE CASCADE,
    series_key        TEXT NOT NULL,

```



```

period_start          DATE NOT NULL,
predicted_value       NUMERIC(20, 4) NOT NULL,
lower_bound          NUMERIC(20, 4),
upper_bound          NUMERIC(20, 4),

PRIMARY KEY (
    forecast_run_id,
    series_key,
    period_start
),

CHECK (
    lower_bound IS NULL
    OR upper_bound IS NULL
    OR lower_bound <= predicted_value
    AND predicted_value <= upper_bound
)
);

CREATE INDEX idx_forecast_values_series_period
    ON forecast_values (series_key, period_start);

```

Correctness conditions

- A forecast run must point to an immutable model version.
- A series can have only one prediction per run and period.
- Confidence bounds must be ordered.
- The forecast's `as_of_date` must be preserved to prevent accidental use of future information during evaluation.

5.3 Audit events

```

CREATE TABLE audit_events (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    aggregate_type     TEXT NOT NULL,
    aggregate_id       UUID NOT NULL,
    event_type         TEXT NOT NULL,
    actor_id           UUID,
    correlation_id      UUID,
    event_data          JSONB NOT NULL DEFAULT '{} '::jsonb,
    occurred_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_aggregate
    ON audit_events (
        tenant_id,
        aggregate_type,
        aggregate_id,
        occurred_at
    );

```

```
CREATE INDEX idx_audit_events_correlation
ON audit_events (correlation_id)
WHERE correlation_id IS NOT NULL;
```

Audit records should normally be append-only.

Do not update an old audit record to describe a new event. Insert a new event.

6. Keys, constraints, indexes, and transactions

6.1 Primary keys

A primary key provides:

- Row identity.
- Uniqueness.
- Non-null enforcement.
- A default index in PostgreSQL.

It does not necessarily represent a business identifier.

6.2 Foreign keys

Foreign keys prevent invalid relationships.

```
model_version_id UUID NOT NULL
REFERENCES model_versions(id)
```

Without the foreign key, a prediction might reference a model version that does not exist.

Delete behaviour

Choose deliberately:

- `ON DELETE RESTRICT`: prevent deletion when dependent rows exist.
- `ON DELETE CASCADE`: delete dependent rows.
- `ON DELETE SET NULL`: retain the child but remove the reference.

Use cascade carefully for finance, audit, and compliance data. Accidental parent deletion can remove large amounts of history.

6.3 Check constraints

Application validation is not sufficient because:

- More than one application may write to the database.
- Bugs can bypass validation.
- Manual migrations or scripts can create invalid records.

Examples:

```
CHECK (amount >= 0)
```

```
CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED'))
```

```
CHECK (lower_bound <= upper_bound)
```

6.4 Index selection

An index should support a real access pattern.

For this query:

```
SELECT *  
FROM prediction_requests  
WHERE tenant_id = $1  
      AND status = 'PENDING'  
ORDER BY created_at  
LIMIT 100;
```

A suitable index is:

```
CREATE INDEX idx_prediction_pending_queue  
ON prediction_requests (  
    tenant_id,  
    status,  
    created_at  
);
```

Composite-index ordering

For a B-tree index:

```
(tenant_id, status, created_at)
```

PostgreSQL can efficiently use the leading columns:

- tenant_id
- tenant_id, status
- tenant_id, status, created_at

It is generally less useful for filtering only on `created_at`.

Partial indexes

When only a small subset is queried frequently:

```
CREATE INDEX idx_pending_predictions
  ON prediction_requests (tenant_id, created_at)
  WHERE status = 'PENDING';
```

This can be smaller and more selective than indexing every status.

Other useful PostgreSQL index types

Index	Typical use
B-tree	Equality, ranges, sorting
GIN	JSONB containment, arrays, full-text search
BRIN	Very large time-ordered tables
GiST	Geospatial and specialized search
Hash	Equality only; B-tree is usually still the default

6.5 Transactions

A transaction protects a set of operations that must succeed or fail together.

Example approval decision:

1. Mark the current approval step approved.
2. Advance the request to the next step.
3. Insert an audit event.
4. Commit all three operations.

If the audit insert fails, the state transition should normally roll back.

6.6 Isolation levels

Read Committed

PostgreSQL's usual default.

Each statement sees committed data at the start of that statement.

Good for:

- Normal CRUD operations.
- Most APIs.
- Short state changes using row locks or optimistic locking.

Potential issue:

- The same query can return different results later in the same transaction.

Repeatable Read

The transaction sees a stable snapshot.

Good for:

- Multi-query reports that must use a consistent snapshot.
- Some reconciliation operations.

Potential issue:

- Serialization failures can occur and require retries.

Serializable

Provides behaviour equivalent to transactions executing sequentially.

Good for:

- High-value financial invariants.
- Complex concurrent decisions that cannot be protected easily by constraints or row locks.

Trade-off:

- More retries.
- Greater contention.
- Higher implementation complexity.

Senior-level answer

Use the weakest isolation level that still protects the business invariant. Add constraints, atomic updates, or targeted locks before making every transaction serializable.

6.7 Optimistic locking

Optimistic locking works well when conflicts are uncommon.

Add:

```
version INTEGER NOT NULL DEFAULT 1
```

Then update:

```
UPDATE approval_requests
SET state = 'APPROVED',
    version = version + 1,
    updated_at = now()
WHERE id = $1
AND version = $2;
```

Correctness condition:

- Exactly one row must be updated.
- Zero updated rows means the record changed after it was read.

The application should return a conflict response, usually HTTP 409, or re-read and retry where safe.

7. Practical task A — Budget versus actual

7.1 Thought process

We need to support:

- Budgets by department, account, and month.
- Actual financial transactions.
- Comparisons where either the budget or actual side is missing.
- Duplicate invoice detection.
- Reconciliation against bank transactions.
- Period-over-period analysis.
- Ranking and rolling totals.

Important decisions:

1. Store budgets at an explicit grain: `tenant + month + department + account`.
2. Store actual transactions at transaction level.
3. Use `NUMERIC`, not floating-point.
4. Keep external transaction IDs for idempotency.
5. Normalize invoice numbers for matching, while preserving the original value.
6. Aggregate actual transactions at query time or into a materialized summary when volume becomes high.

7.2 Pseudocode

```
For every tenant, month, department, and account:
    aggregate budget amount
    aggregate posted actual transactions

    combine both sides
    if either side is absent:
        use zero for reporting
    variance = actual - budget

    if budget is zero:
        variance percentage is undefined
    otherwise:
        variance percentage = variance / budget * 100
```

7.3 Schema

```
CREATE TABLE departments (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    department_code   TEXT NOT NULL,
```

```

        department_name TEXT NOT NULL,

        UNIQUE (tenant_id, department_code)
    );

CREATE TABLE financial_accounts (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    account_code      TEXT NOT NULL,
    account_name      TEXT NOT NULL,

    UNIQUE (tenant_id, account_code)
);

CREATE TABLE budget_lines (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    fiscal_month      DATE NOT NULL,
    department_id     UUID NOT NULL
                        REFERENCES departments(id),
    account_id        UUID NOT NULL
                        REFERENCES financial_accounts(id),
    budget_amount     NUMERIC(18, 2) NOT NULL
                        CHECK (budget_amount >= 0),
    version           INTEGER NOT NULL DEFAULT 1,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (
        tenant_id,
        fiscal_month,
        department_id,
        account_id
    ),

    CHECK (
        fiscal_month =
        date_trunc('month', fiscal_month)::date
    )
);

CREATE TABLE actual_transactions (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    external_transaction_id TEXT NOT NULL,
    department_id     UUID NOT NULL
                        REFERENCES departments(id),
    account_id        UUID NOT NULL
                        REFERENCES financial_accounts(id),
    vendor_id         TEXT,
    invoice_number     TEXT,
    normalized_invoice_number TEXT,
    posting_date       DATE NOT NULL,
    amount            NUMERIC(18, 2) NOT NULL,

```

```

currency_code          CHAR(3) NOT NULL,
status                 TEXT NOT NULL
                        CHECK (status IN (
                            'PENDING',
                            'POSTED',
                            'REVERSED'
                        )),
created_at              TIMESTAMPTZ NOT NULL DEFAULT now(),

UNIQUE (tenant_id, external_transaction_id)
);

CREATE INDEX idx_budget_reporting
ON budget_lines (
    tenant_id,
    fiscal_month,
    department_id,
    account_id
);

CREATE INDEX idx_actual_reporting
ON actual_transactions (
    tenant_id,
    posting_date,
    department_id,
    account_id
)
WHERE status = 'POSTED';

CREATE INDEX idx_actual_invoice_detection
ON actual_transactions (
    tenant_id,
    vendor_id,
    normalized_invoice_number,
    amount
)
WHERE status <> 'REVERSED';

```

7.4 Budget-versus-actual query

```

WITH budget AS (
    SELECT
        tenant_id,
        fiscal_month AS month_start,
        department_id,
        account_id,
        SUM(budget_amount) AS budget_amount
    FROM budget_lines
    WHERE tenant_id = $1
        AND fiscal_month BETWEEN $2 AND $3
    GROUP BY
        tenant_id,

```



```

        fiscal_month,
        department_id,
        account_id
    ),
    actual AS (
        SELECT
            tenant_id,
            date_trunc('month', posting_date)::date AS month_start,
            department_id,
            account_id,
            SUM(amount) AS actual_amount
        FROM actual_transactions
        WHERE tenant_id = $1
            AND posting_date >= $2
            AND posting_date < ($3 + INTERVAL '1 month')
            AND status = 'POSTED'
        GROUP BY
            tenant_id,
            date_trunc('month', posting_date)::date,
            department_id,
            account_id
    )
SELECT
    COALESCE(b.tenant_id, a.tenant_id) AS tenant_id,
    COALESCE(b.month_start, a.month_start) AS month_start,
    COALESCE(b.department_id, a.department_id) AS department_id,
    COALESCE(b.account_id, a.account_id) AS account_id,
    COALESCE(b.budget_amount, 0) AS budget_amount,
    COALESCE(a.actual_amount, 0) AS actual_amount,
    COALESCE(a.actual_amount, 0)
        - COALESCE(b.budget_amount, 0) AS variance_amount,
    CASE
        WHEN COALESCE(b.budget_amount, 0) = 0 THEN NULL
        ELSE ROUND(
            (
                COALESCE(a.actual_amount, 0)
                - b.budget_amount
            ) / b.budget_amount * 100,
            2
        )
    END AS variance_percentage
FROM budget b
FULL OUTER JOIN actual a
    ON a.tenant_id = b.tenant_id
    AND a.month_start = b.month_start
    AND a.department_id = b.department_id
    AND a.account_id = b.account_id
ORDER BY
    month_start,
    department_id,
    account_id;

```

Non-obvious logic

A `FULL OUTER JOIN` is intentional.

An inner join would hide:

- Actual spending with no budget.
- Budget lines with no actual spending.

Those missing-side conditions are often exactly the exceptions finance teams need to identify.

Variance percentage is `NULL` when the budget is zero. Returning `0%` would incorrectly imply no variance.

7.5 Ranking the largest overspending departments

```
WITH department_variance AS (  
    SELECT  
        b.tenant_id,  
        b.fiscal_month,  
        b.department_id,  
        SUM(b.budget_amount) AS budget_amount,  
        COALESCE(SUM(a.amount), 0) AS actual_amount  
    FROM budget_lines b  
    LEFT JOIN actual_transactions a  
        ON a.tenant_id = b.tenant_id  
        AND a.department_id = b.department_id  
        AND a.account_id = b.account_id  
        AND date_trunc('month', a.posting_date)::date = b.fiscal_month  
        AND a.status = 'POSTED'  
    WHERE b.tenant_id = $1  
        AND b.fiscal_month = $2  
    GROUP BY  
        b.tenant_id,  
        b.fiscal_month,  
        b.department_id  
)  
SELECT  
    *,  
    actual_amount - budget_amount AS variance_amount,  
    RANK() OVER (  
        ORDER BY actual_amount - budget_amount DESC  
    ) AS overspend_rank  
FROM department_variance  
ORDER BY overspend_rank;
```

RANK VERSUS ROW_NUMBER

- `RANK()` gives equal rank to ties and leaves gaps.
- `DENSE_RANK()` gives equal rank without gaps.
- `ROW_NUMBER()` assigns a unique sequence even when values tie.

Use `RANK()` when equal overspending should produce equal business rank.

7.6 Rolling three-month actual total

```
WITH monthly_actual AS (  
    SELECT  
        tenant_id,  
        department_id,  
        date_trunc('month', posting_date)::date AS month_start,  
        SUM(amount) AS actual_amount  
    FROM actual_transactions  
    WHERE tenant_id = $1  
        AND status = 'POSTED'  
    GROUP BY  
        tenant_id,  
        department_id,  
        date_trunc('month', posting_date)::date  
)  
SELECT  
    tenant_id,  
    department_id,  
    month_start,  
    actual_amount,  
    SUM(actual_amount) OVER (  
        PARTITION BY tenant_id, department_id  
        ORDER BY month_start  
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
    ) AS rolling_three_month_actual  
FROM monthly_actual  
ORDER BY department_id, month_start;
```

Correctness warning

This is a rolling total over the previous three **rows**, not necessarily three calendar months.

If a department has no row for February, the window might include January, March, and April rather than February, March, and April.

For strict calendar windows, generate a complete month series and left join the actuals.

7.7 Period-over-period comparison

```
WITH monthly_actual AS (  
    SELECT  
        tenant_id,  
        department_id,  
        date_trunc('month', posting_date)::date AS month_start,  
        SUM(amount) AS actual_amount  
    FROM actual_transactions  
    WHERE tenant_id = $1  
        AND status = 'POSTED'  
    GROUP BY
```

```

        tenant_id,
        department_id,
        date_trunc('month', posting_date)::date
    ),
    with_previous AS (
        SELECT
            *,
            LAG(actual_amount) OVER (
                PARTITION BY tenant_id, department_id
                ORDER BY month_start
            ) AS previous_month_actual
        FROM monthly_actual
    )
    SELECT
        *,
        actual_amount - previous_month_actual AS absolute_change,
        CASE
            WHEN previous_month_actual IS NULL
            OR previous_month_actual = 0
            THEN NULL
            ELSE ROUND(
                (
                    actual_amount - previous_month_actual
                ) / previous_month_actual * 100,
                2
            )
            END AS percentage_change
    FROM with_previous;

```

7.8 Duplicate invoice detection

```

SELECT
    tenant_id,
    vendor_id,
    normalized_invoice_number,
    amount,
    currency_code,
    COUNT(*) AS duplicate_count,
    ARRAY_AGG(id ORDER BY created_at) AS transaction_ids
FROM actual_transactions
WHERE tenant_id = $1
    AND status <> 'REVERSED'
    AND normalized_invoice_number IS NOT NULL
GROUP BY
    tenant_id,
    vendor_id,
    normalized_invoice_number,
    amount,
    currency_code
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;

```

Production trade-off

Matching only on invoice number may create false positives because:

- Different vendors may use the same invoice number.
- The same invoice number may be reused across years.
- Formatting may differ: INV-001, INV001, and inv 001.

A practical detection key might include:

```
tenant + vendor + normalized invoice + currency + amount
```

This identifies candidates, not guaranteed fraud. A human or reconciliation workflow should review them.

7.9 Deduplicating imported rows

Suppose repeated ingestion created multiple versions of the same external transaction.

```
WITH ranked AS (  
    SELECT  
        *,  
        ROW_NUMBER() OVER (  
            PARTITION BY tenant_id, external_transaction_id  
            ORDER BY created_at DESC, id DESC  
        ) AS row_number  
    FROM staging_actual_transactions  
)  
SELECT *  
FROM ranked  
WHERE row_number = 1;
```

Correctness depends on defining which row wins. Here, the newest ingestion wins.

That policy should be explicit rather than relying on arbitrary database order.

7.10 Reconciliation schema and exception report

```
CREATE TABLE bank_transactions (  
    id                UUID PRIMARY KEY,  
    tenant_id         UUID NOT NULL,  
    bank_reference     TEXT NOT NULL,  
    transaction_date   DATE NOT NULL,  
    amount            NUMERIC(18, 2) NOT NULL,  
    currency_code      CHAR(3) NOT NULL,  
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    UNIQUE (tenant_id, bank_reference)  
);
```

Exception report:

```
SELECT
    a.id AS ledger_transaction_id,
    a.external_transaction_id,
    a.posting_date,
    a.amount AS ledger_amount,
    b.id AS bank_transaction_id,
    b.amount AS bank_amount,
    CASE
        WHEN b.id IS NULL THEN 'MISSING_IN_BANK'
        WHEN a.amount <> b.amount THEN 'AMOUNT_MISMATCH'
        WHEN a.currency_code <> b.currency_code THEN 'CURRENCY_MISMATCH'
        ELSE 'MATCHED'
    END AS reconciliation_status
FROM actual_transactions a
LEFT JOIN bank_transactions b
    ON b.tenant_id = a.tenant_id
    AND b.bank_reference = a.external_transaction_id
WHERE a.tenant_id = $1
    AND a.status = 'POSTED'
    AND (
        b.id IS NULL
        OR a.amount <> b.amount
        OR a.currency_code <> b.currency_code
    );
```

For real payment reconciliation, matching may require:

- Exact reference matching.
- Date tolerance.
- Amount tolerance.
- One-to-many settlement matching.
- Fees or exchange-rate adjustments.
- Manual override records.

8. Practical task B — Approval workflow

8.1 Thought process

An approval request is not merely a row with an `approved` Boolean.

We need:

- Current overall state.
- Ordered approval steps.
- Approver identity.
- Decision history.
- Concurrency protection.

- Auditability.
- A clear state transition policy.

A multi-stage workflow could be:

```
DRAFT → PENDING → APPROVED
                ↘ REJECTED
                ↘ CANCELLED
```

Each approval step could be:

```
WAITING → PENDING → APPROVED
                ↘ REJECTED
                ↘ SKIPPED
```

8.2 Correctness conditions

- Only the current pending step can be decided.
- A rejected request cannot later become approved without an explicit restart.
- The same step cannot be approved twice.
- All required steps must succeed before the request becomes approved.
- Every decision must produce an audit event.
- Concurrent decisions must not overwrite one another.

8.3 Pseudocode

```
begin transaction

load or atomically update request using expected version

if the request version changed:
    return conflict

verify request is PENDING
verify selected step is the current step
verify selected step is still PENDING
verify actor is authorized

update step with decision

if rejected:
    set request state to REJECTED
else if another required step exists:
    mark next step PENDING
    increment current step number
else:
    set request state to APPROVED

increment request version
insert audit event
```

commit transaction

8.4 Schema

```
CREATE TABLE approval_requests (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    entity_type       TEXT NOT NULL,
    entity_id         UUID NOT NULL,
    state             TEXT NOT NULL
                    CHECK (state IN (
                        'DRAFT',
                        'PENDING',
                        'APPROVED',
                        'REJECTED',
                        'CANCELLED'
                    )),
    current_step_number INTEGER,
    requested_by      UUID NOT NULL,
    requested_at      TIMESTAMPTZ,
    version           INTEGER NOT NULL DEFAULT 1,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (
        tenant_id,
        entity_type,
        entity_id
    ),

    CHECK (
        state = 'DRAFT'
        OR requested_at IS NOT NULL
    )
);

CREATE TABLE approval_steps (
    request_id        UUID NOT NULL
                    REFERENCES approval_requests(id)
                    ON DELETE CASCADE,
    step_number       INTEGER NOT NULL
                    CHECK (step_number > 0),
    approver_id       UUID NOT NULL,
    status            TEXT NOT NULL
                    CHECK (status IN (
                        'WAITING',
                        'PENDING',
                        'APPROVED',
                        'REJECTED',
                        'SKIPPED'
                    )),
    decision_comment  TEXT,
```



```

    decided_at          TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),

    PRIMARY KEY (request_id, step_number),

    CHECK (
        (
            status IN ('APPROVED', 'REJECTED')
            AND decided_at IS NOT NULL
        )
        OR
        (
            status NOT IN ('APPROVED', 'REJECTED')
            AND decided_at IS NULL
        )
    )
);

CREATE INDEX idx_pending_approvals
    ON approval_steps (approver_id, created_at)
    WHERE status = 'PENDING';

CREATE INDEX idx_approval_entity
    ON approval_requests (
        tenant_id,
        entity_type,
        entity_id
    );

```

8.5 Optimistic-locking transition

First, claim the request version:

```

UPDATE approval_requests
SET
    version = version + 1,
    updated_at = now()
WHERE id = $1
    AND tenant_id = $2
    AND version = $3
    AND state = 'PENDING';

```

The application must verify that one row was updated.

Then decide the step:

```

UPDATE approval_steps
SET
    status = $4,
    decision_comment = $5,
    decided_at = now()
WHERE request_id = $1

```

```
AND step_number = $6
AND status = 'PENDING';
```

Again, exactly one row must be updated.

All statements must execute inside the same database transaction.

Failure mode

Do not:

1. Update the approval step.
2. Commit.
3. Update the request.
4. Commit.
5. Insert audit history.

A crash after step 1 could leave the request and step inconsistent.

8.6 Pending approvals query

```
SELECT
    r.id AS request_id,
    r.entity_type,
    r.entity_id,
    r.current_step_number,
    s.approver_id,
    r.requested_at,
    now() - r.requested_at AS waiting_duration
FROM approval_requests r
JOIN approval_steps s
    ON s.request_id = r.id
   AND s.step_number = r.current_step_number
WHERE r.tenant_id = $1
   AND r.state = 'PENDING'
   AND s.status = 'PENDING'
ORDER BY r.requested_at;
```

8.7 Approval history query

```
SELECT
    r.id AS request_id,
    s.step_number,
    s.approver_id,
    s.status,
    s.decision_comment,
    s.decided_at,
    LAG(s.decided_at) OVER (
        PARTITION BY r.id
        ORDER BY s.step_number
```

```
        ) AS previous_decision_time
FROM approval_requests r
JOIN approval_steps s
    ON s.request_id = r.id
WHERE r.tenant_id = $1
    AND r.id = $2
ORDER BY s.step_number;
```

8.8 Pessimistic locking alternative

You can lock the request row:

```
SELECT *
FROM approval_requests
WHERE id = $1
FOR UPDATE;
```

This blocks concurrent transactions attempting to lock the same row.

Use pessimistic locking when:

- Conflicts are frequent.
- The operation is short.
- Waiting is acceptable.

Use optimistic locking when:

- Conflicts are rare.
- Requests can be edited across longer user interactions.
- You prefer returning a conflict over holding locks.

Never hold a database transaction open while waiting for a user, external API, or model response.

9. Practical task C — RAG document catalog

9.1 Source-of-truth separation

A vector database should normally not be the only record of document state.

A robust design uses:

Object storage

Stores:

- Original PDF or document.
- Parsed text artifacts.
- Extracted tables or images.
- Large intermediate files.

PostgreSQL

Stores:

- Stable document ID.
- Tenant ownership.
- Document version history.
- Checksums.
- Processing status.
- Chunk IDs.
- Parser and embedding versions.
- Audit and lineage.

Vector database

Stores:

- Embedding vectors.
- Vector record IDs.
- Metadata required for filtering.
- References to stable PostgreSQL chunk IDs.

The vector index should be rebuildable from authoritative metadata and source content.

9.2 Thought process

We need to distinguish:

- A logical document.
- Individual document versions.
- Chunks generated from one version.
- The embedding model used.
- The vector-store record corresponding to each chunk.
- Ingestion jobs and failures.

A content checksum enables deduplication and idempotent ingestion.

9.3 Pseudocode

```
receive upload
calculate checksum

store source bytes in object storage

begin database transaction
  find logical document
  if the same checksum already exists:
    return existing version

  create document version with status PENDING
```

```

        create ingestion job
        create outbox event
    commit transaction

worker reads event
download object
parse and chunk content

store chunk metadata in PostgreSQL
write embeddings into vector store

after every required vector write succeeds:
    mark chunks READY
    mark version INDEXED
    mark version active

if any operation fails:
    mark job FAILED
    retain enough information for retry

```

9.4 Schema

```

CREATE TABLE documents (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    logical_key       TEXT NOT NULL,
    title             TEXT NOT NULL,
    source_type       TEXT NOT NULL,
    status            TEXT NOT NULL
                    CHECK (status IN (
                        'ACTIVE',
                        'ARCHIVED',
                        'DELETED'
                    )),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (tenant_id, logical_key)
);

CREATE TABLE document_versions (
    id                UUID PRIMARY KEY,
    document_id       UUID NOT NULL
                    REFERENCES documents(id),
    version_number    INTEGER NOT NULL
                    CHECK (version_number > 0),
    content_uri       TEXT NOT NULL,
    content_sha256    TEXT NOT NULL,
    mime_type         TEXT NOT NULL,
    parser_version    TEXT NOT NULL,
    chunking_config   JSONB NOT NULL,
    processing_status TEXT NOT NULL

```

```

        CHECK (processing_status IN (
            'PENDING',
            'PARSING',
            'CHUNKED',
            'INDEXING',
            'INDEXED',
            'FAILED'
        )),
    is_active          BOOLEAN NOT NULL DEFAULT false,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (document_id, version_number),
    UNIQUE (document_id, content_sha256)
);

CREATE UNIQUE INDEX idx_one_active_document_version
    ON document_versions (document_id)
    WHERE is_active = true;

CREATE TABLE document_chunks (
    id                UUID PRIMARY KEY,
    document_version_id    UUID NOT NULL
                        REFERENCES document_versions(id)
                        ON DELETE CASCADE,
    chunk_index        INTEGER NOT NULL
                        CHECK (chunk_index >= 0),
    text_content        TEXT NOT NULL,
    text_sha256         TEXT NOT NULL,
    token_count         INTEGER
                        CHECK (token_count >= 0),
    metadata            JSONB NOT NULL DEFAULT '{} '::jsonb,
    embedding_model_version_id    UUID
                        REFERENCES model_versions(id),
    vector_record_id    TEXT,
    vector_status        TEXT NOT NULL
                        CHECK (vector_status IN (
                            'PENDING',
                            'READY',
                            'FAILED',
                            'DELETED'
                        ))
    ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (document_version_id, chunk_index)
);

CREATE INDEX idx_chunks_version
    ON document_chunks (
        document_version_id,
        chunk_index
    );

CREATE INDEX idx_chunks_metadata_gin
    ON document_chunks

```

```

        USING GIN (metadata);

CREATE TABLE ingestion_jobs (
    id                UUID PRIMARY KEY,
    tenant_id         UUID NOT NULL,
    document_version_id UUID NOT NULL
                        REFERENCES document_versions(id),
    status            TEXT NOT NULL
                        CHECK (status IN (
                            'PENDING',
                            'RUNNING',
                            'SUCCEEDED',
                            'FAILED'
                        )),
    attempt_count      INTEGER NOT NULL DEFAULT 0
                        CHECK (attempt_count >= 0),
    error_code         TEXT,
    error_message      TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

9.5 Active document-version query

```

SELECT
    d.id AS document_id,
    d.tenant_id,
    d.title,
    dv.id AS version_id,
    dv.version_number,
    dv.content_uri,
    dv.content_sha256,
    dv.parser_version,
    dv.chunking_config,
    dv.processing_status
FROM documents d
JOIN document_versions dv
    ON dv.document_id = d.id
   AND dv.is_active = true
WHERE d.tenant_id = $1
      AND d.id = $2
      AND d.status = 'ACTIVE';

```

The tenant predicate is mandatory. Fetching only by document ID can create a cross-tenant security vulnerability.

9.6 Detecting stale embeddings

Suppose the production embedding model is \$3.

```

SELECT
    dc.id AS chunk_id,
    dc.document_version_id,
    dc.embedding_model_version_id,
    dc.vector_status
FROM document_chunks dc
JOIN document_versions dv
    ON dv.id = dc.document_version_id
JOIN documents d
    ON d.id = dv.document_id
WHERE d.tenant_id = $1
    AND dv.is_active = true
    AND (
        dc.vector_status <> 'READY'
        OR dc.embedding_model_version_id IS DISTINCT FROM $3
    );

```

IS DISTINCT FROM is useful because it handles NULL safely.

Unlike:

```
embedding_model_version_id <> $3
```

it also returns rows where embedding_model_version_id is NULL.

9.7 Avoiding dual-write inconsistency

A dangerous flow is:

1. Commit PostgreSQL metadata.
2. Write to vector storage.
3. Application crashes before updating PostgreSQL status.

Now the systems disagree.

An outbox table helps:

```

CREATE TABLE outbox_events (
    id                UUID PRIMARY KEY,
    aggregate_type    TEXT NOT NULL,
    aggregate_id      UUID NOT NULL,
    event_type        TEXT NOT NULL,
    payload            JSONB NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at      TIMESTAMPTZ
);

CREATE INDEX idx_unpublished_outbox
    ON outbox_events (created_at)
    WHERE published_at IS NULL;

```


The metadata update and outbox insert happen in one PostgreSQL transaction.

A publisher later sends the event to the worker system and marks it published.

This does not make PostgreSQL and the vector database one atomic transaction. It makes retries and recovery explicit.

Required idempotency

Vector writes should use a deterministic identifier such as:

```
tenant_id:document_version_id:chunk_index:embedding_model_version
```

Retrying the write then replaces or confirms the same vector record instead of creating duplicates.

10. Analytical SQL concepts

10.1 Joins

Inner join

Returns only matching records.

```
FROM budget_lines b
JOIN departments d
  ON d.id = b.department_id
```

Left join

Preserves every row from the left side.

Useful for exception detection:

```
FROM actual_transactions a
LEFT JOIN bank_transactions b
  ON ...
WHERE b.id IS NULL
```

Full outer join

Preserves unmatched records from both sides.

Useful for budget-versus-actual comparisons.

Cross join

Produces combinations.

Useful when creating a complete grid of:

```
months × departments × accounts
```

Be careful because result size grows multiplicatively.

10.2 Common table expressions

CTEs improve readability:

```
WITH monthly_actual AS (...),  
with_previous AS (...)  
SELECT ...
```

They are useful for:

- Breaking complex transformations into named steps.
- Recursive queries.
- Pre-aggregation.
- Deduplication.
- Analytical pipelines.

Do not assume a CTE automatically improves performance. Modern PostgreSQL may inline non-recursive CTEs, but materialization choices still matter.

10.3 Window functions

Window functions preserve row-level detail while computing values across related rows.

Examples:

```
SUM(amount) OVER (...)
```

```
LAG(amount) OVER (...)
```

```
ROW_NUMBER() OVER (...)
```

```
RANK() OVER (...)
```

The key difference from `GROUP BY`:

- `GROUP BY` collapses rows.
 - A window function retains the rows.
-

11. Query plans and performance diagnosis

11.1 Starting command

```
EXPLAIN (  
    ANALYZE,
```

```
BUFFERS,  
VERBOSE  
)  
SELECT ...
```

`ANALYZE` actually executes the query.

For destructive statements, do not casually run:

```
EXPLAIN ANALYZE DELETE ...
```

against production data. Use a safe environment or wrap carefully in a transaction that is rolled back.

11.2 What to inspect

Estimated versus actual rows

Example problem:

```
estimated rows: 10  
actual rows: 500,000
```

The planner selected a strategy using incorrect cardinality estimates.

Possible causes:

- Stale statistics.
- Correlated columns.
- Highly skewed values.
- Complex predicates.
- Functions applied to indexed columns.

Possible actions:

```
ANALYZE actual_transactions;
```

or increase statistics for important columns.

Sequential scan

A sequential scan is not automatically bad.

It may be correct when:

- Most table rows are needed.
- The table is small.
- The filter is not selective.
- Reading the index and then many table pages costs more.

A sequential scan is suspicious when a query returns a tiny fraction of a large table and a suitable index should exist.

Nested-loop join

Good when:

- The outer result is small.
- The inner side has an efficient index.

Bad when:

- The outer result is unexpectedly large.
 - The inner side is repeatedly scanned.
-

Hash join

Often good for joining larger unordered datasets using equality.

Watch for memory pressure and disk spilling.

Sort operation

Look for:

- Large sort input.
 - External merge or disk spill.
 - Repeated sorts that an index could avoid.
-

Buffers

`BUFFERS` shows whether pages came from:

- Shared cache.
- Disk reads.
- Temporary files.

High execution time with many buffer reads can indicate excessive I/O.

11.3 Functions on indexed columns

This query may prevent a normal index on `posting_date` from being used efficiently:

```
WHERE date_trunc('month', posting_date) = DATE '2026-08-01'
```

Prefer a range:

```
WHERE posting_date >= DATE '2026-08-01'
AND posting_date < DATE '2026-09-01'
```

Alternatively, use an expression index when that expression is a common query pattern.

11.4 N+1 query problem

Suppose the application loads 100 documents:

```
SELECT * FROM documents LIMIT 100;
```

Then makes another query for every document:

```
SELECT *  
FROM document_versions  
WHERE document_id = $1  
AND is_active = true;
```

Total:

```
1 document query + 100 version queries = 101 queries
```

Solutions:

- Join the active version in one query.
- Batch with `WHERE document_id = ANY($1)`.
- Use ORM eager loading.
- Use a data-loader pattern.
- Precompute a read model when necessary.

The problem is not merely query count. It also causes:

- Network round-trip overhead.
 - Connection-pool pressure.
 - Higher tail latency.
 - Database CPU overhead.
 - Inconsistent snapshots across calls.
-

12. Redis caching

12.1 Cache-aside

The application controls the cache.

```
read cache  
if hit:  
    return cached value  
  
read database
```

```
write value to cache  
return value
```

On update:

```
write database  
invalidate cache
```

This is common because it keeps PostgreSQL authoritative.

Strengths

- Simple.
- Cache only useful records.
- Cache failure does not need to block database access.

Weaknesses

- First request is slow.
- Stale data can exist.
- Cache invalidation must be implemented correctly.

12.2 Read-through

The application requests data through a cache abstraction, and the cache layer loads missing values from the database.

Conceptually:

```
application → cache loader → database
```

This can simplify callers, but the loading library or infrastructure becomes more complex.

12.3 Write-through

Writes are sent through the cache layer, which updates both cache and persistent storage.

Potential advantage:

- Cached data remains warm.

Major concern:

- What happens when the cache write succeeds but the database write fails?
- What happens when the database succeeds but cache propagation fails?

The ordering and failure semantics must be explicit.

12.4 TTL

TTL limits how long a cached record can remain stale.

Examples:

Data	Example TTL
Document catalog metadata	1–10 minutes
Model configuration	30–300 seconds
Negative lookup	5–30 seconds
Expensive retrieval result	30–300 seconds
Static reference data	Hours
Approval state	Usually short TTL or explicit invalidation

TTL should be based on business staleness tolerance, not selected arbitrarily.

12.5 Invalidation

Common strategies:

Delete on write

After the database commit:

```
DEL document cache key
```

The next read repopulates it.

Versioned keys

```
rag:document:v5:{tenant}:{document_id}
```

When the representation changes, increment `v5` to `v6`.

This is particularly useful when deploying incompatible serialization changes.

Event-driven invalidation

A committed database event triggers cache invalidation.

Use when multiple services maintain caches.

Short TTL only

Acceptable when:

- Small staleness is tolerable.
 - Updates are infrequent.
 - Event-driven invalidation is not worth the complexity.
-

12.6 Cache stampede

A stampede occurs when a popular key expires and many requests query the database simultaneously.

Prevention techniques:

- Distributed lock or single-flight request.
- TTL jitter.
- Stale-while-revalidate.
- Refresh before expiry.
- Local request coalescing.
- Prewarming.
- Negative caching for repeated misses.

TTL jitter

Instead of every key expiring after exactly 300 seconds:

```
TTL = 300 + random(0, 60)
```

This avoids many related keys expiring simultaneously.

13. Cache key design

A cache key must encode every factor that changes the result.

13.1 Document metadata

```
rag:document:v1:{tenant_id}:{document_id}:active
```

13.2 Retrieval results

```
rag:retrieval:v3:
{tenant_id}:
{index_version}:
{embedding_model_version}:
{normalized_query_hash}:
{filter_hash}:
{top_k}
```

A retrieval cache must include:

- Tenant.
- Query.
- Filters.
- Embedding model.
- Index or corpus version.
- top_k.

- Permissions or access scope where relevant.

Omitting filters can return results for the wrong department, business unit, or user.

13.3 Prompt cache

```
prompt:v2:
{tenant_id}:
{prompt_template_version}:
{model_version}:
{normalized_input_hash}
```

Also include relevant generation settings:

- Temperature.
- Maximum output tokens.
- Tool configuration.
- System prompt version.
- Structured-output schema version.

A non-deterministic model response may be inappropriate to cache unless the product accepts repeated output.

13.4 Model-output cache

```
model-output:v4:
{tenant_id}:
{model_version}:
{input_hash}:
{parameter_hash}
```

Do not cache only by raw user message. The same message under a different system prompt or model version may require a different answer.

13.5 Security condition

Tenant identity must be inside the key and must also be validated in the underlying database query.

Cache key isolation is not a substitute for database authorization.

14. Repository-plus-cache Python example

This example uses:

- PostgreSQL as the source of truth.
- Redis cache-aside.

- Tenant-aware keys.
- Short negative caching.
- TTL jitter.
- A lightweight distributed lock for stampede reduction.
- Safe lock release using a token.

14.1 Thought process

The repository abstraction should hide storage details from the service layer.

We separate:

```
PostgresDocumentRepository
    ↓
CachedDocumentRepository
    ↓
Application service
```

The cache wrapper should:

1. Check Redis.
2. Decode and validate the cached representation.
3. Return a cached missing sentinel when appropriate.
4. Acquire a short lock on a cache miss.
5. Recheck the cache after obtaining the lock.
6. Load from PostgreSQL.
7. Cache the result with jitter.
8. Release only the lock it owns.
9. Fall back safely if Redis is unavailable.

14.2 Pseudocode

```
function get_active_document(tenant, document):
    key = tenant-aware cache key

    cached = redis.get(key)
    if cached represents a document:
        return document
    if cached represents missing:
        return none

    try acquire short-lived lock

    if lock acquired:
        recheck cache

        record = postgres.get_active_document(...)

        if record exists:
            cache serialized record with TTL and jitter
```

```

        else:
            cache missing sentinel for short TTL

        safely release lock
        return record

    briefly wait
    retry cache once

    if still missing:
        query postgres directly

```

14.3 Code

```

from __future__ import annotations

import asyncio
import json
import random
import secrets
from dataclasses import asdict, dataclass
from typing import Protocol
from uuid import UUID

import asyncpg
from redis.asyncio import Redis
from redis.exceptions import RedisError

@dataclass(frozen=True, slots=True)
class ActiveDocument:
    document_id: UUID
    tenant_id: UUID
    title: str
    version_id: UUID
    version_number: int
    content_uri: str
    content_sha256: str
    parser_version: str
    processing_status: str

    def to_json(self) -> str:
        payload = asdict(self)
        payload["document_id"] = str(self.document_id)
        payload["tenant_id"] = str(self.tenant_id)
        payload["version_id"] = str(self.version_id)
        return json.dumps(payload, separators=(",", ":"))

    @classmethod
    def from_json(cls, value: str) -> "ActiveDocument":
        payload = json.loads(value)

        return cls(

```

```

        document_id=UUID(payload["document_id"]),
        tenant_id=UUID(payload["tenant_id"]),
        title=payload["title"],
        version_id=UUID(payload["version_id"]),
        version_number=int(payload["version_number"]),
        content_uri=payload["content_uri"],
        content_sha256=payload["content_sha256"],
        parser_version=payload["parser_version"],
        processing_status=payload["processing_status"],
    )

```

```

class DocumentRepository(Protocol):
    async def get_active_document(
        self,
        tenant_id: UUID,
        document_id: UUID,
    ) -> ActiveDocument | None:
        ...

```

```

class PostgresDocumentRepository:
    def __init__(self, pool: asyncpg.Pool) -> None:
        self._pool = pool

```

```

    async def get_active_document(
        self,
        tenant_id: UUID,
        document_id: UUID,
    ) -> ActiveDocument | None:
        query = """
            SELECT
                d.id AS document_id,
                d.tenant_id,
                d.title,
                dv.id AS version_id,
                dv.version_number,
                dv.content_uri,
                dv.content_sha256,
                dv.parser_version,
                dv.processing_status
            FROM documents d
            JOIN document_versions dv
              ON dv.document_id = d.id
            AND dv.is_active = true
            WHERE d.tenant_id = $1
            AND d.id = $2
            AND d.status = 'ACTIVE'
        """

```

```

    async with self._pool.acquire() as connection:
        row = await connection.fetchrow(
            query,
            tenant_id,

```

```

        document_id,
    )

    if row is None:
        return None

    return ActiveDocument(
        document_id=row["document_id"],
        tenant_id=row["tenant_id"],
        title=row["title"],
        version_id=row["version_id"],
        version_number=row["version_number"],
        content_uri=row["content_uri"],
        content_sha256=row["content_sha256"],
        parser_version=row["parser_version"],
        processing_status=row["processing_status"],
    )

class CachedDocumentRepository:
    _MISSING_VALUE = '{"missing":true}'

    _RELEASE_LOCK_SCRIPT = """
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("del", KEYS[1])
        end
        return 0
    """

    def __init__(
        self,
        repository: DocumentRepository,
        redis: Redis,
        *,
        base_ttl_seconds: int = 300,
        ttl_jitter_seconds: int = 60,
        negative_ttl_seconds: int = 15,
        lock_ttl_seconds: int = 5,
    ) -> None:
        self._repository = repository
        self._redis = redis
        self._base_ttl_seconds = base_ttl_seconds
        self._ttl_jitter_seconds = ttl_jitter_seconds
        self._negative_ttl_seconds = negative_ttl_seconds
        self._lock_ttl_seconds = lock_ttl_seconds

    @staticmethod
    def _cache_key(
        tenant_id: UUID,
        document_id: UUID,
    ) -> str:
        return (
            f"rag:document:v1:"
            f"{tenant_id}:{document_id}:active"

```

```

    )

    @staticmethod
    def _lock_key(cache_key: str) -> str:
        return f"{cache_key}:lock"

    async def _read_cache(
        self,
        cache_key: str,
    ) -> tuple[bool, ActiveDocument | None]:
        """
        Returns:
            (True, document) when a cached document exists.
            (True, None) when a negative-cache entry exists.
            (False, None) on cache miss or Redis failure.
        """
        try:
            value = await self._redis.get(cache_key)
        except RedisError:
            return False, None

        if value is None:
            return False, None

        if isinstance(value, bytes):
            value = value.decode("utf-8")

        if value == self._MISSING_VALUE:
            return True, None

        try:
            return True, ActiveDocument.from_json(value)
        except (KeyError, TypeError, ValueError, json.JSONDecodeError):
            # Corrupt or incompatible cache data should not break reads.
            try:
                await self._redis.delete(cache_key)
            except RedisError:
                pass
            return False, None

    async def _populate_cache(
        self,
        cache_key: str,
        document: ActiveDocument | None,
    ) -> None:
        try:
            if document is None:
                await self._redis.set(
                    cache_key,
                    self._MISSING_VALUE,
                    ex=self._negative_ttl_seconds,
                )
            return

```

```

        ttl = (
            self._base_ttl_seconds
            + random.randint(0, self._ttl_jitter_seconds)
        )

        await self._redis.set(
            cache_key,
            document.to_json(),
            ex=ttl,
        )
    except RedisError:
        # PostgreSQL remains authoritative.
        pass

async def _release_lock(
    self,
    lock_key: str,
    lock_token: str,
) -> None:
    try:
        await self._redis.eval(
            self._RELEASE_LOCK_SCRIPT,
            1,
            lock_key,
            lock_token,
        )
    except RedisError:
        pass

async def get_active_document(
    self,
    tenant_id: UUID,
    document_id: UUID,
) -> ActiveDocument | None:
    cache_key = self._cache_key(
        tenant_id,
        document_id,
    )

    found, cached_document = await self._read_cache(
        cache_key
    )

    if found:
        return cached_document

    lock_key = self._lock_key(cache_key)
    lock_token = secrets.token_urlsafe(18)

    try:
        lock_acquired = bool(
            await self._redis.set(
                lock_key,
                lock_token,

```

```

        ex=self._lock_ttl_seconds,
        nx=True,
    )
)
except RedisError:
    lock_acquired = False

if lock_acquired:
    try:
        # Another process may have populated the cache
        # between our first GET and lock acquisition.
        found, cached_document = await self._read_cache(
            cache_key
        )
        if found:
            return cached_document

        document = (
            await self._repository.get_active_document(
                tenant_id,
                document_id,
            )
        )

        await self._populate_cache(
            cache_key,
            document,
        )
        return document
    finally:
        await self._release_lock(
            lock_key,
            lock_token,
        )

# Another request may currently be loading the same key.
await asyncio.sleep(0.05)

found, cached_document = await self._read_cache(
    cache_key
)
if found:
    return cached_document

# Availability is preferred over waiting indefinitely.
return await self._repository.get_active_document(
    tenant_id,
    document_id,
)

async def invalidate(
    self,
    tenant_id: UUID,
    document_id: UUID,

```



```
) -> None:
    cache_key = self._cache_key(
        tenant_id,
        document_id,
    )

    try:
        await self._redis.delete(cache_key)
    except RedisError:
        pass
```

14.4 Non-obvious correctness details

Tenant-aware cache key

This prevents two tenants with the same document ID from sharing a key.

The SQL still validates `tenant_id`; both layers enforce isolation.

Negative caching

Repeated requests for a nonexistent document can overload the database.

A short missing-record TTL reduces repeated misses without hiding newly created data for too long.

Lock token

Deleting a lock unconditionally is unsafe.

Example:

1. Request A obtains a five-second lock.
2. Request A becomes slow.
3. Lock expires.
4. Request B obtains a new lock.
5. Request A finishes and executes `DEL lock`.
6. Request A accidentally deletes Request B's lock.

The Lua script deletes the lock only when the stored token matches Request A's token.

Redis failure

Redis is treated as an optimization. A Redis failure falls back to PostgreSQL.

This is often preferable to making a metadata API unavailable because its cache is down.

Remaining limitation

The lock TTL must exceed the expected repository load time. If database loading takes longer than the lock TTL, more than one request may still access PostgreSQL.

A production system may add:

- Lock renewal.

- Local single-flight coordination.
 - Bounded retry with exponential backoff.
 - Stale-while-revalidate.
 - Metrics for lock contention and cache load latency.
-

15. Consistency across database, cache, object storage, and vector store

PostgreSQL and Redis

Typical guarantee:

- PostgreSQL is authoritative.
- Redis may be temporarily stale.
- Database updates commit first.
- Cache invalidation happens after commit.

Failure mode:

```
database commit succeeds  
cache invalidation fails
```

Mitigations:

- Short TTL.
 - Retry through an outbox event.
 - Versioned cache keys.
 - Event-driven invalidation.
-

PostgreSQL and object storage

An object-store upload and database transaction cannot normally share one ACID transaction.

Safer workflow:

1. Upload object using a unique temporary key.
2. Verify checksum.
3. Commit metadata referencing the object.
4. Mark the object ready or move/copy it to the final logical location.
5. Periodically delete orphaned temporary objects.

Alternative:

1. Create metadata as `PENDING_UPLOAD`.
2. Upload the object.
3. Mark metadata as `UPLOADED`.

Both approaches require cleanup and retry handling.

PostgreSQL and vector store

Treat vector indexing as an asynchronous projection.

Useful states:

```
PENDING → INDEXING → INDEXED
                ↘ FAILED
```

A document should become searchable only after required chunks are indexed successfully.

For partial failure:

- Record failed chunk IDs.
- Retry idempotently.
- Do not silently mark the full document ready.
- Consider whether partial retrieval is acceptable for the product.

Object storage and vector store

Do not derive document identity from a temporary file path.

Use stable IDs from PostgreSQL:

```
document_id
document_version_id
chunk_id
```

The vector store should contain these IDs as metadata so retrieval results can be validated against current relational state.

16. Important production failure modes

Over-indexing

Every index:

- Consumes storage.
- Slows inserts and updates.
- Increases vacuum and maintenance work.
- Can complicate planner choices.

Create indexes from access patterns and query-plan evidence.

Missing tenant filter

This is both a correctness and security problem.

Bad:

```
SELECT *
FROM documents
WHERE id = $1;
```

Better:

```
SELECT *
FROM documents
WHERE tenant_id = $1
AND id = $2;
```

PostgreSQL row-level security can provide another protection layer, but application queries should still be explicit.

Caching authorization-sensitive results

A retrieval result may depend on:

- Tenant.
- User.
- Department.
- Security groups.
- Document access version.

A key that omits authorization scope may leak restricted information.

Caching failed model responses

Do not cache transient failures as successful output.

If failures are cached at all, use:

- A distinct error representation.
 - Very short TTL.
 - Explicit retryability classification.
-

Unbounded JSONB

JSONB is useful but can become a dumping ground.

Promote a JSON field into a typed column when it is:

- Frequently filtered.
- Frequently joined.
- Required.
- Constrained.

- Used for sorting or aggregation.
-

Mutable model versions

Once a model version is used for predictions, its configuration and artifact reference should be immutable.

Changing the record in place destroys reproducibility.

Register a new model version instead.

Long-running transactions

Long transactions:

- Hold locks.
- Prevent cleanup of old row versions.
- Increase table bloat.
- Create contention.

Do not keep transactions open during:

- LLM calls.
 - File uploads.
 - Human approvals.
 - External HTTP requests.
 - Vector embedding computation.
-

17. Senior interview decision framework

When asked, “Which database would you choose?”, answer in this order:

- 1. Define the access patterns.**
 - Point reads, range queries, joins, analytics, writes, scans.
- 2. Define consistency requirements.**
 - Strong consistency, eventual consistency, multi-record atomicity.
- 3. Define scale and distribution.**
 - Data size, throughput, tenant count, hot-key risks, regional needs.
- 4. Define query evolution.**
 - Fixed access patterns versus frequent ad hoc reporting.
- 5. Define operational constraints.**
 - Managed service, team expertise, backups, disaster recovery, cost.
- 6. Select the simplest system satisfying the requirements.**

A strong answer might be:

I would use PostgreSQL as the source of truth for approvals and financial records because we need multi-row transactions, constraints, reconciliation queries, and auditability. I would add Redis as a cache for repeated document metadata or retrieval results, object storage for original files, and a vector store as a rebuildable semantic-search index. I would consider DynamoDB for extremely high-throughput job or session state only if the access patterns were stable and key-oriented.

18. Interview drill

1. Why is PostgreSQL often preferred for approval workflows?

Because approval workflows require state transitions, relational constraints, transactions, auditability, and concurrency protection.

2. Why should a vector database not be the only document catalog?

Vector databases optimize similarity search, not authoritative lifecycle, versioning, transactional metadata, audit, or cross-entity consistency.

3. What is optimistic locking?

A concurrency technique that updates a row only when its version matches the version previously read. Zero affected rows indicate a conflict.

4. When is a sequential scan acceptable?

When the table is small or the query needs a large percentage of its rows, making an index scan more expensive.

5. What causes an N+1 query problem?

Loading a collection with one query and issuing another query for each item. It causes excessive round trips and database load.

6. How do you prevent cache stampede?

Use single-flight or distributed locks, TTL jitter, stale-while-revalidate, prefetching, and short negative caching.

7. Why use `FULL OUTER JOIN` for budget versus actual?

To retain budget-only and actual-only records, both of which may represent important exceptions.

8. Why is `NUMERIC` preferred over `FLOAT` for money?

`NUMERIC` represents decimal values exactly, while binary floating-point introduces rounding errors.

9. What is the outbox pattern?

A business-state update and an event are inserted in the same database transaction. A separate publisher reliably processes unpublished events.

10. How do you choose a composite-index order?

Start from common equality predicates, then range or sorting columns, while considering selectivity and actual query patterns.

19. Day 3 completion checklist

You should now be able to explain and implement:

- PostgreSQL versus DynamoDB versus MongoDB selection.
- Source-of-truth separation across SQL, Redis, object storage, and vector stores.
- Transactional approval workflows.
- Optimistic and pessimistic concurrency control.
- Model, prediction, forecast, document, and audit schemas.
- Budget-versus-actual reporting.
- Reconciliation and duplicate invoice detection.
- CTEs, joins, window functions, ranking, rolling totals, and deduplication.
- Query-plan analysis and N+1 diagnosis.
- Tenant-safe cache keys.
- Cache-aside with TTL, negative caching, invalidation, and stampede protection.
- Outbox-based recovery for cross-system workflows.

Day 3 DSA — HashMap / Dictionary

1. Beginner-friendly summary

A hash map stores values using keys:

```
employee_salary["E101"] = 120000
```

It is useful when you need fast:

- Counting.
- Grouping.
- Membership checking.
- Mapping a value to its position.
- Looking up a required complement.

In Python, the main hash-map type is `dict`.

In Go, it is `map[KeyType]ValueType`.

Average lookup, insertion, and deletion are **$O(1)$** , although pathological collision cases can degrade performance.

2. Recognition signals

Think of a hash map when the problem includes phrases such as:

Signal in the problem	Likely hash-map pattern
“Count how many times...”	Frequency map
“Group items having the same...”	Grouping map
“Find whether this value already appeared”	Membership lookup
“Return the index of...”	Value-to-index map
“Find two values whose sum is...”	Complement lookup
“Find duplicate elements”	Set or frequency map
“Find subarrays with sum...”	Prefix-sum frequency map
“Avoid nested loops”	Replace repeated searching with a map
“Process in one pass”	Store previously seen information

A useful interview question is:

What information from previously processed elements would let me answer the current step in constant time?

If that information can be stored by a key, a hash map is often appropriate.

3. Core hash-map patterns

3.1 Counting

Count how often each item appears.

```
values = ["approved", "pending", "approved", "rejected"]

frequency: dict[str, int] = {}

for value in values:
    frequency[value] = frequency.get(value, 0) + 1

print(frequency)
```

Result:

```
{
    "approved": 2,
    "pending": 1,
```



```
"rejected": 1
}
```

Typical uses:

- Word frequencies.
 - Duplicate detection.
 - Vote counting.
 - Transaction status counts.
 - Character-frequency problems.
-

3.2 Grouping

Map a derived key to multiple related values.

```
employees = [
    ("Amit", "Finance"),
    ("Neha", "Engineering"),
    ("Ravi", "Finance"),
]

by_department: dict[str, list[str]] = {}

for name, department in employees:
    by_department.setdefault(department, []).append(name)
```

Result:

```
{
    "Finance": ["Amit", "Ravi"],
    "Engineering": ["Neha"]
}
```

Typical uses:

- Group anagrams.
 - Group transactions by account.
 - Group employees by team.
 - Group database rows by tenant.
 - Build adjacency lists for graphs.
-

3.3 Indexing

Store where a value appeared.

```
numbers = [40, 10, 30, 20]

index_by_value: dict[int, int] = {}
```

```
for index, value in enumerate(numbers):  
    index_by_value[value] = index
```

Result:

```
{  
    40: 0,  
    10: 1,  
    30: 2,  
    20: 3  
}
```

Typical uses:

- Return original indexes.
- Track the latest occurrence.
- Detect repeated values.
- Connect IDs to objects.
- Avoid repeatedly scanning a list.

When duplicates matter, store a list of indexes:

```
indexes_by_value: dict[int, list[int]] = {}  
  
for index, value in enumerate(numbers):  
    indexes_by_value.setdefault(value, []).append(index)
```

3.4 Complement lookup

Suppose the target is 10 and the current number is 4.

The required complement is:

```
10 - 4 = 6
```

Instead of searching the remaining array for 6, check whether 6 has already been stored in the map.

This is the core idea behind Two Sum:

```
required = target - current
```

It also appears in prefix-sum problems:

```
required previous prefix = current prefix - target
```

4. Collision intuition

A hash map uses a hash function to convert a key into an internal bucket location.

Conceptually:



Different keys can sometimes produce the same bucket. This is called a **collision**.



Hash-map implementations resolve collisions using mechanisms such as:

- Separate chaining.
- Open addressing.
- Probing.
- Bucket expansion and table resizing.

Interview-level intuition

Hash-map operations are typically:

- Average lookup: $O(1)$
- Average insertion: $O(1)$
- Average deletion: $O(1)$
- Worst-case lookup: $O(n)$

The worst case can occur when many keys collide or the table becomes badly distributed.

You generally do not implement collision handling yourself in interview problems. Python and Go manage it internally.

Keys must be hashable

Python dictionary keys must have a stable hash and equality behaviour.

Valid examples:

```
lookup[10] = "integer"
lookup["model-v2"] = "string"
lookup[(1, 2)] = "tuple"
```

Invalid example:

```
lookup[[1, 2]] = "list"
```

A list is mutable and therefore cannot be used as a dictionary key.

5. Medium problem — Subarray Sum Equals K

Problem statement

Given an integer array `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals `k`.

Example

```
Input:
nums = [1, 1, 1]
k = 2

Output:
2
```

The matching subarrays are:

```
indexes 0..1 → [1, 1]
indexes 1..2 → [1, 1]
```

6. Recognition signals

This problem contains several signals:

- We need the **number of subarrays**, not just one.
- The array can contain negative values.
- Repeated prefix sums matter.
- A nested-loop solution is possible but too slow.
- We need to find whether a related previous sum exists.
- The required previous value is a complement:

```
current_prefix_sum - k
```

This suggests:

```
prefix sum + hash-map frequency counting
```

7. Brute-force reasoning

Approach

Start every possible subarray at index `left`.

For each `left`, expand `right` and maintain the running sum.

```
for every left index:
    sum = 0

    for every right index from left onward:
        add nums[right]

        if sum equals k:
            increment answer
```

Pseudocode

```
answer = 0

for left from 0 to n - 1:
    current_sum = 0

    for right from left to n - 1:
        current_sum += nums[right]

        if current_sum == k:
            answer += 1

return answer
```

Complexity

- Time: $O(n^2)$
- Extra space: $O(1)$

This is correct, but too slow for large arrays.

8. Optimized reasoning

8.1 Prefix-sum idea

Let:

```
prefix[j] = sum of elements from index 0 through j
```

The sum of a subarray from `i + 1` through `j` is:

```
prefix[j] - prefix[i]
```

We want that subarray sum to equal k :

```
prefix[j] - prefix[i] = k
```

Rearrange:

```
prefix[i] = prefix[j] - k
```

Therefore, at every current prefix sum, we need to know:

| How many previous prefix sums equal $\text{current_prefix} - k$?

That is a complement lookup.

8.2 Why store frequencies rather than only existence?

Consider:

```
nums = [0, 0]
k = 0
```

Prefix sum 0 occurs multiple times.

Several previous prefix positions can produce valid subarrays ending at the current index.

Therefore, the map must store:

```
prefix sum → number of times previously seen
```

not merely:

```
prefix sum → True
```

8.3 Why initialize {0: 1}?

Before processing any number, the prefix sum is conceptually zero once.

```
prefix_frequency = {0: 1}
```

This allows subarrays beginning at index 0 to be counted.

Example:

```
nums = [3]
k = 3
```

After reading 3:

```
current_prefix = 3
required_prefix = 3 - 3 = 0
```

Because prefix 0 exists once, [3] is counted.

Without {0: 1}, valid subarrays beginning at the first element would be missed.

9. Optimized pseudocode

```
prefix_frequency = map containing 0 → 1
current_prefix = 0
answer = 0

for each number:
    current_prefix += number

    required_prefix = current_prefix - k

    answer += number of times required_prefix was seen

    increment frequency of current_prefix

return answer
```

The order is important:

1. Count matching previous prefixes.
2. Then add the current prefix to the map.

This ensures the current prefix is not incorrectly treated as a previous prefix.

10. Step-by-step example

```
nums = [1, 2, 1, 2]
k = 3
```

Initial state:

```
frequency = {0: 1}
prefix = 0
answer = 0
```

Current value	Prefix	Required prefix-k	Previous count	Answer
1	1	-2	0	0
2	3	0	1	1
1	4	1	1	2

2	6	3	1	3
---	---	---	---	---

Valid subarrays:

```
[1, 2]
[2, 1]
[1, 2]
```

11. Python solution

```
from collections import defaultdict
from collections.abc import Sequence

def count_subarrays_with_sum(
    nums: Sequence[int],
    target: int,
) -> int:
    """
    Return the number of contiguous subarrays whose sum equals target.

    Time complexity: O(n)
    Space complexity: O(n)
    """
    prefix_frequency: dict[int, int] = defaultdict(int)
    prefix_frequency[0] = 1

    prefix_sum = 0
    subarray_count = 0

    for number in nums:
        prefix_sum += number

        required_prefix = prefix_sum - target
        subarray_count += prefix_frequency[required_prefix]

        prefix_frequency[prefix_sum] += 1

    return subarray_count

def main() -> None:
    test_cases = [
        ([1, 1, 1], 2, 2),
        ([1, 2, 1, 2], 3, 3),
        ([1, -1, 0], 0, 3),
        ([3], 3, 1),
        ([], 0, 0),
        ([0, 0], 0, 3),
    ]
```



```
for nums, target, expected in test_cases:
    actual = count_subarrays_with_sum(nums, target)

    print(
        f"nums={nums}, target={target}, "
        f"expected={expected}, actual={actual}"
    )

    assert actual == expected

if __name__ == "__main__":
    main()
```

12. Non-obvious logic

The map stores previous prefixes

At the start of each iteration, `prefix_frequency` represents prefix sums observed before the current position.

That invariant is why lookup happens before insertion.

Multiple equal prefixes create multiple subarrays

Suppose the required prefix appeared three times.

Then three different starting positions produce valid subarrays ending at the current position.

Therefore:

```
subarray_count += prefix_frequency[required_prefix]
```

not:

```
if required_prefix in prefix_frequency:
    subarray_count += 1
```

Negative numbers are supported

Unlike a standard sliding window, this solution remains correct with negative numbers.

A sliding-window approach depends on predictable monotonic behaviour:

- Expanding the window increases the sum.
- Shrinking the window decreases the sum.

Negative numbers break those assumptions.

Example:

```
[4, -2, 1]
```

Adding a new element can decrease the sum, so normal sliding-window decisions become unreliable.

13. Correctness argument

At each ending position j , let the current prefix sum be P .

A previous prefix sum Q defines a subarray ending at j whose sum is:

```
P - Q
```

The subarray sum equals k exactly when:

```
P - Q = k
```

which means:

```
Q = P - k
```

The hash map stores the number of previous occurrences of every Q .

Therefore, adding:

```
frequency[P - k]
```

counts every valid subarray ending at the current position exactly once.

Repeating this for every ending position counts all valid subarrays.

14. Edge cases

Empty array

```
nums = []
```

There are no non-empty subarrays, so the answer is 0.

Entire array equals target

```
nums = [1, 2, 3]
k = 6
```

The initial $0 \rightarrow 1$ entry makes this count correctly.

Target is zero

```
nums = [1, -1, 0]
k = 0
```

Valid subarrays include:

```
[1, -1]
[0]
[1, -1, 0]
```

Repeated prefix sums must be counted.

Multiple zeros

```
nums = [0, 0]
k = 0
```

Valid subarrays:

```
first [0]
second [0]
[0, 0]
```

Answer: 3.

Negative target

```
nums = [-1, -2, 1]
k = -3
```

The same algorithm works without modification.

Large answer

For many zeros, the number of matching subarrays can grow quadratically.

For n zeros and target zero:

```
answer = n * (n + 1) / 2
```

Python integers expand automatically.

In Go, use `int64` if input constraints can produce answers larger than a 32-bit integer.

15. Complexity

Brute force

- Time: $O(n^2)$
- Extra space: $O(1)$

Optimized prefix map

- Time: $O(n)$ average
- Space: $O(n)$

The map can contain up to $n + 1$ distinct prefix sums.

16. Go solution

```
package main

import (
    "fmt"
)

func countSubarraysWithSum(nums []int, target int64) int64 {
    // Prefix sum 0 has occurred once before processing the array.
    prefixFrequency := map[int64]int64{
        0: 1,
    }

    var prefixSum int64
    var subarrayCount int64

    for _, number := range nums {
        prefixSum += int64(number)

        requiredPrefix := prefixSum - target
        subarrayCount += prefixFrequency[requiredPrefix]

        prefixFrequency[prefixSum]++
    }

    return subarrayCount
}

func main() {
    testCases := []struct {
        nums      []int
        target     int64
        expected  int64
    }{
        {[[]int{1, 1, 1}, 2, 2},
        {[[]int{1, 2, 1, 2}, 3, 3},
        {[[]int{1, -1, 0}, 0, 3},
    }
```

```

    {[[]int{3}, 3, 1},
    {[[]int{}, 0, 0},
    {[[]int{0, 0}, 0, 3},
}

for _, testCase := range testCases {
    actual := countSubarraysWithSum(
        testCase.nums,
        testCase.target,
    )

    fmt.Printf(
        "nums=%v target=%d expected=%d actual=%d\n",
        testCase.nums,
        testCase.target,
        testCase.expected,
        actual,
    )

    if actual != testCase.expected {
        panic("test failed")
    }
}
}

```

17. Python versus Go comparison

Area	Python	Go
Map declaration	<code>dict[int, int]</code>	<code>map[int64]int64</code>
Missing key read	Usually use <code>.get()</code> or <code>defaultdict</code>	Returns the value type's zero value
Increment	<code>freq[key] += 1</code>	<code>freq[key]++</code>
Integer overflow	Integers grow dynamically	Choose <code>int64</code> when counts may be large
Generic readability	Very concise	Explicit types improve production clarity
Concurrency safety	Normal <code>dict</code> is not generally safe for concurrent mutation	Normal <code>map</code> is not safe for concurrent reads/writes

Go-specific backend note

A normal Go map must not be concurrently mutated by multiple goroutines without synchronization.

Possible protection:

```

var mu sync.RWMutex
frequency := make(map[string]int)

```

or use `sync.Map` for suitable specialized access patterns.

For this interview algorithm, concurrency is unnecessary. Adding goroutines would increase complexity without improving the $O(n)$ dependency chain.

18. Common mistakes

Mistake 1: Storing only existence

Incorrect:

```
seen_prefixes: set[int]
```

A set loses how many times each prefix occurred.

Use:

```
prefix_frequency: dict[int, int]
```

Mistake 2: Forgetting the initial zero prefix

Incorrect:

```
prefix_frequency = {}
```

Correct:

```
prefix_frequency = {0: 1}
```

Mistake 3: Updating before lookup

Risky order:

```
prefix_frequency[prefix_sum] += 1
answer += prefix_frequency[prefix_sum - target]
```

Correct order:

```
answer += prefix_frequency[prefix_sum - target]
prefix_frequency[prefix_sum] += 1
```

Mistake 4: Using sliding window with negative numbers

Sliding window is generally not reliable here because array values may be negative.

Mistake 5: Returning a Boolean

The problem asks for the total number of subarrays, so every matching previous prefix must contribute to the answer.

19. Interview-ready explanation

I first considered an $O(n^2)$ solution that fixes each starting position and expands the ending position while maintaining a running sum. To optimize it, I use prefix sums. If the current prefix is P , a previous prefix must equal $P-k$ for the intervening subarray to sum to k . I store the frequency of every previous prefix in a hash map, initialized with $\{0: 1\}$ so subarrays beginning at index zero are counted. For each number, I update the prefix, add the frequency of $\text{prefix}-k$ to the result, and then record the current prefix. This gives average $O(n)$ time and $O(n)$ space and works with negative numbers.

20. Day 3 DSA checklist

You should now be able to explain:

- Frequency counting.
- Grouping by a derived key.
- Value-to-index mapping.
- Complement lookup.
- Hash collision intuition.
- Average versus worst-case hash-map complexity.
- Prefix sums with frequency maps.
- Why $\{0: 1\}$ is necessary.
- Why repeated prefix sums require counts.
- Why ordinary sliding windows fail with negative values.
- The Python and Go implementations of Subarray Sum Equals K.